KYB First-Line Review Agent with Approval Gates — Design Document

AI Engineering Cohort 2 · Capstone · Hareesh Vethurajan (individual) · avrhareesh@gmail.com · 30 Oct 2026

1. Problem statement

Merchant onboarding (KYB) review is a manual bottleneck: for each applicant an analyst reads the application (registry data, ultimate beneficial owners, documents, declared activity), looks up the relevant rules in a long onboarding policy and decides **approve**, **request more information** or **escalate to compliance**. The error cost is asymmetric — wrongly approving a high-risk merchant is far worse than wrongly escalating a low-risk one.

The system reviews **one merchant application** and returns exactly one of those three decisions plus the policy clauses that justify it, as `{decision, cited_clauses, reasoning, missing_items, risk_tier}`. "Approve" is a proposal only: the tool that activates a merchant is gated behind explicit human confirmation, enforced in the tool-execution layer rather than by the prompt. The narrow question answered: **does a tool-using agent with policy retrieval beat (a) one LLM call with the whole policy in context and (b) vanilla RAG — on per-class decision accuracy, missed escalations, citation groundedness and wrong-action rate — and is the extra cost justified?** Out of scope: real customer data, post-activation monitoring, event streaming. Inspired by reference idea #13, adapted to a compliance domain with retrieval and a three-way decision.

2. Architecture

Data surface. (1) A 25-clause onboarding policy written from public AML/KYB guidance, chunked one clause per chunk with metadata (`clause_id`, `section`, `entity_type`, `risk_tier`), embedded locally with fastembed `bge-small-en-v1.5`, stored in Chroma (cosine). (2) Lookup tools over a synthetic world: `lookup_registry`, `screen_sanctions`, `check_documents`. (3) Action tools: `request_info`, `escalate`, `propose_approval` → `activate_merchant` (gated).

Retrieval design. Metadata pre-filter on `entity_type`, then dense top-k over clause chunks (k = 6); variant B+ adds BM25 + reciprocal rank fusion (k = 60). Retrieved clause IDs are logged per run so retrieval and generation are scored separately.

Three approaches, built in this order and compared on the same 50 applications:

- **A — Baseline:** one LLM call, full policy + application + pre-fetched lookups, structured output. No retrieval, no tools.
- **B / B+ — Vanilla RAG (workflow):** lookups pre-fetched in a fixed order, clauses retrieved as above, one LLM call. The flow is fixed; only the output is probabilistic.
- **C — ReAct agent (LangGraph):** the LLM chooses lookups and retrievals in a bounded loop (max 8 model / 12 tool calls, one tool per turn); every tool takes a public `reason`; every step is traced (tool, args, output, latency, tokens). Approval gate = LangGraph `interrupt_before=["activate"]`; `activate_merchant` refuses without an HMAC-signed, application-bound, expiring, single-use token that only the human-confirmation step can mint.

Cross-cutting. Pydantic schemas for inputs, tool arguments and the decision record (invalid output → one retry, then escalate). Input guardrail: schema validation and out-of-scope check before any LLM call. Context budget: application ≤ 2k tokens, clauses ≤ 1.5k, tool outputs head/tail-truncated at 800. Reviewer prompt with an ordered review procedure (hard stops → risk tier → completeness → approve); application text and tool outputs declared untrusted. Generator `gpt-4.1-mini` (the answer is in the context; no reasoning model). Judge `gpt-4.1`. Non-conversational by design: one application per run, bounded loop, so no compaction or memory layer is needed.

3. Evaluation criteria

Dataset. 50 synthetic applications (20 approve / 15 request-info / 15 escalate) from a committed, seeded generator; **gold decisions and gold clause IDs are computed by a deterministic rule checker written from the policy**, then hand-verified by the author (merchant-onboarding experience at a payments company). The LLM only reads rendered application text; labels are objective and reproducible. 10 adversarial variants with injected instructions in the applicant's free text. Each approach is run 3×; results are reported as counts and percentages (one case = 2 accuracy points).

Quantitative: decision accuracy (exact match); per-class precision/recall with **missed escalations (target 0/15 per run)** as the headline; citation precision/recall (cited ∈ gold clauses); tool-call correctness for C (required calls present + ordering constraints, e.g. `check_documents` before the final action); **wrong-action rate = 0** (any `activate_merchant` without a valid token, across approved / rejected / timed-out / tampered / replay tests, enforced by pytest); tokens, cost and latency per application from traces.

Qualitative: binary LLM-as-judge for groundedness ("is the reasoning supported only by the cited clauses, application fields and lookup outputs?") and completeness of `missing_items`; one judge per metric, rubric-based, stronger model than the generator, each judgement run with the evidence in two orders and passed only if both agree; human–judge agreement, TPR and TNR on 20 hand-labelled reviews (bar: ≥ 0.9 before the judge is trusted).

Decision rule: the final architecture must beat the cheaper alternative on accuracy and missed escalations by a margin that survives the cost/latency difference; otherwise the simpler approach is chosen. The README failure-analysis section records every pivot.

4. Framework justification

- **LangGraph** for the agent: explicit graph state and conditional edges give control over the loop, and its checkpointing plus interrupt is a clean fit for a human approval gate; it is the control end of the autonomy spectrum and common in production. Pydantic AI's tool-approval flow would also work; LangGraph was chosen for explicit state and familiarity. smolagents defaults to code-executing agents, unnecessary here and harder to sandbox.
- **Pydantic** for schemas; **Chroma** (local, cosine) — small corpus, no server; BM25 only in the hybrid variant; **OpenAI gpt-4.1-mini** for generation via the cohort key, **gpt-4.1** as judge; **pytest** for gate tests. The A → B → C comparison and the eval harness are plain Python, so the agent framework is swappable without touching the evals.

5. What changed between the one-pager and the delivered system

Planned (one-pager, 26 Oct)	Delivered (30 Oct)	Why
Generator `gpt-4o-mini`	`gpt-4.1-mini`	On a 5-app smoke test 4o-mini missed both escalations and misread day counts; 4.1-mini with an ordered review procedure in the prompt fixed it. Tuning stopped there to avoid fitting the eval set.
Judge: Claude (different model family)	`gpt-4.1`	No Anthropic key available; the stronger-OpenAI fallback named in the one-pager was used.
Judge trusted at ≥ 0.9 human agreement	Measured **0.70** on groundedness	Below the bar; the cause (judge sees only cited clauses) is documented and the groundedness pass rates are reported as indicative only.

Hybrid retrieval only if evals show misses	Built and reported as B+	Dense recall of gold clauses was 0.12; hybrid lifted it to 0.42.
Optional Gradio UI	Not built	Time; the approval flow is exercised by the LangGraph interrupt and the gate tests, and shown in the demo video via the CLI and traces.
“Final choice by the decision rule”	**A, the stuffed-context baseline**	93.3 % accuracy and 4/45 missed escalations vs 80–83 % and 14–15/45 for RAG and the agent, at a third of the agent’s cost. The policy fits in context; retrieval and agency only removed information.